
120 minutes. Tous Documents papier autorisés. Les appareils électroniques ne sont pas autorisés.

Liste des exercices et Barème à titre indicatif.

	Page
1 [Esterel] Régulateur de vitesse	1
2 [Lustre] Accélération	2
3 [Web] Rendez-vous de vaccination	2
4 [Canaux Synchrones] Fête de la musique	4

1 [Esterel] Régulateur de vitesse

On souhaite modéliser un régulateur de vitesse en Esterel.

Le conducteur indique au régulateur la vitesse souhaitée en fonction des panneaux de signalisation qu'il observe.

Un observateur observe la vitesse réelle à l'aide d'un capteur et la transmet au régulateur.

Le régulateur reçoit les informations transmises par le conducteur et l'observateur en parallèle. L'accélération est mise à jour à la fin de l'instant en fonction des signaux reçus:

- Si le conducteur et l'observateur envoient un signal, alors
 $acc = (WantedSpeed - RealSpeed) * 3600$
- Si seul le conducteur envoie un signal, alors
 $acc = (WantedSpeed - RegSpeed) * 3600$
- Si seul l'observateur envoie un signal, alors
 $acc = (RegSpeed - RealSpeed) * 3600$

La vitesse est mise à jour à la fin de l'instant en fonction de l'accélération, en supposant qu'un instant correspond à une seconde:

$RegSpeed := RegSpeed + acc$

1.1 Donner le code du module conducteur. On pourra utiliser des fonctions `RandTime` et `RandSpeed` (sans donner leur code) pour simuler, respectivement, le temps en instants entre deux panneaux successifs et la valeur de la limitation de vitesse d'un panneau;

1.2 Donner le code du module observateur.

- 1.3 Donner le code du module régulateur en gérant les interactions avec les deux modules précédent à l'aide de deux branches en parallèle.
- 1.4 Donner le code du module permettant de modéliser le problème dans son ensemble (il devra notamment exécuter les trois modules précédents et gérer entre autres les signaux de fin).
- 1.5 Donner des traces d'exécutions possible mettant en valeur les différents comportement demandé dans l'énoncé.

2 [Lustre] Accélération

On considère un chariot placé sur un rail qui avance ou recul.

On note X sa position, V sa vitesse et A son accélération.

Au premier instant, X vaut 0m et V vaut 1 m/s .

L'accélération A est un flot d'entrée de type `real` qui donne sa valeur une fois au début de chaque seconde.

On rappelle que l'accélération est la dérivée de la vitesse et que la vitesse est la dérivée de la position. Aussi, on peut utiliser le taux d'accroissement pour les approximer. Ainsi, si h est assez petit, alors on a à tout instant t :

- $V(t+h)-V(t) = A(t)*h$
- $X(t+h)-X(t) = V(t)*h$

2.1 Donner le typage d'un programme lustre `cart` modélisant le problème. Vous donnerez votre réponse sous la forme d'un nœud à compléter.

2.2 Définir un programme Lustre ayant pour sortie le flot de type `real` X .

2.1 On suppose que le flot d'accélération vaut:

$$1; -2; 3; -1; 6; 2$$

Donner dans un tableau les valeurs des trois flots A , V et X .

3 [Web] Rendez-vous de vaccination

Le gouvernement d'un pays met en place une **application de réservation de créneaux de vaccination** répartie sur deux plate-formes. La **plate-forme usager** gère les interactions avec les utilisateurs qui souhaitent réserver un créneau de vaccination alors que la **plate-forme centre** gère le contact avec les centres de vaccination et peut être contactée par la plate-forme usager pour mettre à jour ses données.

La plate-forme usager offre les fonctionnalités suivantes:

- les usagers peuvent **s'enregistrer** et **se connecter** avec un couple d'identifiants login/mot de passe,
- les usagers peuvent **afficher** la liste des **créneaux** de vaccination **disponibles** dans les prochains jours, par centre, dans l'ordre de proximité du centre par rapport à la position de l'utilisateur,
- les usagers peuvent réserver **un créneau** de vaccination parmi ceux proposés, **cette opération rend le créneau inaccessible aux autres usagers**.

- les usagers peuvent **annuler** une réservation.
- quand un usager se connecte au site, les créneaux de réservation choisis précédemment lui sont **rappelés**.

La plate-forme centre met à disposition les fonctionnalités suivantes:

- **ouverture créneaux:** un centre de vaccination envoie directement à la plate-forme une liste de nouveaux créneaux disponibles.
- **nouvelles réservations:** un centre de vaccination C demande à la plateforme la liste des créneaux de C qui ont été réservés depuis le dernier appel à cette fonctionnalité.
- **mise à jour:** la plate-forme usager contacte la plate-forme centre en lui joignant la liste de toutes les réservations prises depuis le dernier appel à cette fonctionnalité, et reçoit en retour la liste des nouveaux créneaux mis à disposition depuis le dernier appel à cette fonctionnalité

Le but de cet exercice est de réaliser un mini-dossier décrivant l'implémentation de cette application web en respectant les points suivants:

- l'application est composée d'un client pour les usager, d'un client (léger) pour les centres, d'un serveur pour la plate-forme usager et d'un serveur pour la plate-forme centre.
- Les serveurs Web des deux plate-formes sont écrits en Servlet purs ou en Go (sans utiliser de *framework* de développement).
- Le serveur de la plate-forme usager et son client communiquent selon une API REST.
- La serveur plate-forme centre communiquent avec son client et avec le serveur de la plate-forme usager (vu comme client pour cette interaction) par le Web selon l'*Approche Service* (SOAP, RPC ou directement).

Le dossier peut comprendre:

1. **Schéma:** un **schéma général** de l'application, détaillant les composants internes coté client et côté serveur, les différentes communications entre les composantes de l'application, et une répartition des fonctionnalités.
2. **Use Cases:** une série de plusieurs *use cases* décrivant **textuellement** (en prose) des exemples d'utilisations caractéristiques de l'application.
3. **DB:** une description des bases de données des deux plate-formes, des différentes tables/-documents et de leur composition ainsi que quelques exemples d'entrées.
4. **REST:** une description complète de l'API REST mise en oeuvre par la plate-forme principale, des mots-clefs utilisés, des requêtes possibles, ainsi que plusieurs exemples d'utilisation (requete HTTP, reponse HTTP).
5. **Services:** donner une description **semi-formelle** (mais pas WSDL) des services offerts par la plate-forme centre et de leur utilisation, ainsi qu'un paragraphe expliquant à quel moment, et dans quel but est appelé chacun des services.
6. **Servlets/Handlers:** un plan (noms, signatures, rôles) des Servlets/Handlers présents dans les serveurs de l'application, de leurs liens avec les mots-clef de l'API ou les services.
7. **Servlet:** le plan du code d'un Servlet ou d'un *handler* Go caractéristique, au choix.
8. **Client:** le plan du site pour les usager avec les différents écrans proposés (on peut choisir un client monopage), et une description des contenus proposés et la mise en avant des appels asynchrones (AJAX) effectués par le client.
(on ne demande pas de plan pour le client de la plate-forme centre)

4 [Canaux Synchrones] Fête de la musique

On décide d'implémenter un programme permettant à plusieurs musiciens distants bénéficiant de connexions internet aux *ping* (ou *round-trip time*) hétérogènes de jouer ensemble.

Le principe est le suivant: chaque musicien j enchaîne des motifs musicaux $m_0^j, m_1^j, m_2^j, \dots$, chacun d'une durée (en mesures) fixe d déterminée et commune à tout le monde. Quand un motif m_i^j (le motif numéro i joué par le musicien j) est terminé, il est envoyé (on ne s'intéresse pas à l'encodage et la sérialisation de la musique dans cet exercice) à un serveur, qui rassemble tous les motifs envoyés par tous les musiciens, et envoie à chaque musicien une superposition des motifs **des autres musiciens** M_i^j (la superposition des motifs numéro i joués par tout le monde sauf j).

Le musicien j reçoit ainsi M_i^j pendant qu'il est en train de jouer le motif m_{i+1}^j (on suppose initialement que le rapport entre le ping des participants et la durée fixée est suffisamment petit pour que ce soit toujours le cas) et l'application joue (par les haut-parleurs de j) le motif M_i^j en le synchronisant sur le début du motif m_{i+2}^j .

Ainsi chaque musicien, de son point de vue, joue avec un décalage de $2d$ avec les autres musiciens.

On dispose d'un type abstrait `motif` et des fonctions suivantes:

- en *OCaml*

```
recupere_motif : int -> motif
joue_motif : motif -> int -> unit
superpose_motifs : motif list -> motif
```

- en *Go*:

```
recupere_motif(j int) motif {...}
joue_motif(m motif, j int) {...}
superpose_motifs(m []motif) motif {...}
```

`recupere_motif` récupère le dernier motif (on suppose que la durée d est codée en dur dans la fonction) joué devant son micro par le musicien j . Elle est bloquante tant qu'un motif entier n'a pas été joué.

`joue_motif` joue dans les haut-parleurs du musicien j un motif musical en entier au début de son prochain motif joué (la fonction s'occupe de la synchronisation). Elle n'est pas bloquante.

`superpose_motif` rassemble plusieurs motifs en un seul.

- 4.1 On suppose initialement que tous les musiciens sont gérés par la même machine, et on veut créer un thread (une goroutine) pour gérer les entrées-sorties (à l'aide des fonctions ci-dessus) de chaque musicien (et bien sûr un thread - une goroutine - pour le serveur).
Faire un schéma du système en indiquant les différentes entités et les canaux de communication (avec leurs types).
- 4.2 Donner le code du système en supposant que tous les musiciens sont connectés avec un *ping* raisonnable et jouent à la même vitesse.
- 4.3 Améliorer le système en permettant à un musicien de jouer ses motifs plus vite, ou de ne pas jouer de motif, sans poser de problème aux autres musiciens.
- 4.4 Expliciter les modifications de code à réaliser à votre réponse à la question 4.2 pour que les musiciens et le serveur soient gérés par des machines distantes, en utilisant internet pour communiquer en suivant un schéma client-serveur classique.